

Relatório de Análise Técnica e Refatoração

Disciplina: Engenharia de Software

Instituição: PUC Minas

Trabalho: Detecção de Bad Smells e Refatoração Segura

Aluno: João Paulo Gobira

Github: <https://github.com/Gobiraa/bad-smells-js-refactoring>

Matrícula: 859611

1. Análise de Smells

Durante a revisão manual do arquivo `src/ReportGenerator.js`, foram identificados três *Bad Smells* clássicos que comprometem o design da aplicação:

- **Método Longo (Long Method):** A função `generateReport` centralizava múltiplas responsabilidades — ela definia cabeçalhos, avaliava permissões de usuários (ADMIN vs. USER), aplicava lógicas de negócio (prioridade de valores) e formatava a saída para CSV e HTML. Qualquer mudança na regra de negócio ou no layout de exportação exigiria a alteração do mesmo método, violando o Princípio da Responsabilidade Única (SRP). Testar esse método se tornou complexo, exigindo a cobertura de dezenas de ramificações diferentes em um único bloco de código.
- **Código Duplicado (Duplicated Code):** A lógica de iteração sobre os itens, incremento do valor total e a montagem das strings repetia-se quase integralmente dentro dos blocos condicionais de ADMIN e USER. Se um novo campo precisasse ser adicionado ao relatório, o desenvolvedor teria que lembrar de alterar o código em vários lugares, aumentando o risco de inconsistências.
- **Condicionais Repetidas (Repeated Switches / Ifs):** O código verificava constantemente `if (reportType === 'CSV') ... else if (reportType === 'HTML')` no cabeçalho, no corpo e no rodapé do relatório. Isso violava o Princípio do Aberto/Fechado (OCP). Adicionar um novo formato, como PDF ou JSON, exigiria modificar a classe existente espalhando novos blocos condicionais por todo o método principal.

2. Relatório da Ferramenta

A utilização do `eslint` aliado ao plugin `sonarjs` foi fundamental para quantificar problemas

estruturais que a análise manual não consegue medir com exatidão. Enquanto a leitura humana indicou que o método era excessivamente extenso, a ferramenta apontou objetivamente a infração de complexidade cognitiva (sonarjs/cognitive-complexity) com uma pontuação de 27, superando o limite estabelecido de 15.

Além disso, a ferramenta detectou a oportunidade de simplificação no fluxo de controle (sonarjs/no-collapsible-if), apontando onde as condicionais aninhadas de regras de usuário poderiam ser mescladas. Isso demonstra como a análise estática complementa a análise manual: o analista identifica problemas semânticos e de domínio (como Números Mágicos e efeitos colaterais de mutabilidade) e a máquina atua como um rastreador preciso para métricas de complexidade e acoplamento estrutural.

```
PS D:\Biblioteca\Documentos\GitHub\bad-smells-js-refactoring> npx eslint src/

D:\Biblioteca\Documentos\GitHub\bad-smells-js-refactoring\src\ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                         sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

3. Processo de Refatoração

O *smell* mais crítico corrigido foi o Método Longo atrelado às Condicionais Repetidas. Para isolar as responsabilidades e preparar a arquitetura do sistema para futuras expansões sem a necessidade de modificação do núcleo do código, aplicou-se a técnica de refatoração Replace Conditional with Polymorphism combinada com o padrão de projeto Strategy, além do Extract Method para segregar as regras de filtragem de domínio.

Antes (Código Original com Bad Smells)	Depois (Arquitetura Proposta com Strategy)
<pre>// Condicionais espalhadas e repetitivas if (reportType === 'CSV') { report += `\${item.id},\${item.name}...`; total += item.value; } else if (reportType === 'HTML') { const style = item.priority ? ... report += `<tr \${style}><td>...`; total += item.value; }</pre>	<pre>// Classes especialistas e polimórficas class CsvReportStrategy { generate(user, items) { ... } } class HtmlReportStrategy { generate(user, items) { ... } } // Classe principal mapeia e delega this.strategies = { 'CSV': new CsvReportStrategy(), 'HTML': new HtmlReportStrategy() }; const strategy = this.strategies[reportType]; return strategy.generate(user, itensFiltrados);</pre>

```
> bad-smells-js-refactoring@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js

(node:19668) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/ReportGenerator.refactored.test.js
PASS tests/ReportGenerator.test.js

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        0.527 s, estimated 1 s
Ran all test suites.
```

4. Conclusão

A execução desta atividade evidenciou a premissa de que a refatoração jamais deve alterar o comportamento observável do sistema de produção. O uso do framework Jest como rede de segurança foi indispensável nesse processo. A manutenção de uma suíte de testes automatizados com 100% de cobertura de sucesso forneceu o respaldo necessário para a realização de modificações arquiteturais profundas com a certeza de que nenhuma regressão ou quebra de contrato com o cliente foi introduzida.

A eliminação sistemática dos gargalos de manutenção e acoplamento complexo resultou em um artefato final limpo, robusto, aderente aos princípios SOLID e altamente extensível para a incorporação de novos requisitos com esforço minimizado.